MonoSQL on DynamoDB Customer Use Case

MonoSQL's intended customer?

Companies who are using RDS database, but their use cases are more suitable for NoSQL database. These use cases include: user event system in Gaming industry, order management and sales system in Retail industry, content meta management in Media industry etc.

Companies who are using DynamoDB but need SQL query API such as Joins, Aggrgration etc. to simplify their dev costs.

The Painpoint of customer

For customer who are using SQL database on NoSQL use case. The painpoints are two folds:

1. RDS database's performance drops significantly as data size increases. The DDL opration on RDS tables takes days to finish and blocks new feature releases.
2. The migration from RDS to NoSQL is not straightforward, which will introduce initial dev cost, ongoing dev cost, infra cost, operation cost and schema migration cost.

For customer who have already adopted DynamoDB, the painpoints includes:

1. Doing Join or Aggrgation at application layer which is costly and prone to bugs.
2. New features or feature iterations still require heavy ongoing dev cost due to using DynamoDB SDK instead of normal SQL interface.

MonoSQL's solution

MonoSQL is the SQL wrapper for AWS DyanmoDB. Customer is enable to migrate from RDS to DynamoDB without modifing their application code. Customer can still use JDBC/ODBC to connect to their database. Customer can still join and aggrgate their data inside database.

MonoSQL's value

The biggest value of MonoSQL is to save cost, improve performance, reduce administrative overhead for customer. MonoSQL acheives these goals by helping customer to move the use case, which is more suitable for NoSQL, from SQL/RDS database to AWS DynamoDB.

Note that SQL database is not a one-size-fits-all solution, even for Oracle. Modern data architecture acknowledges the idea that taking a one-size-fits-all approach will lead to compromises. Follow this architecture AWS offers more than 15 purpose-built data engines to support diverse data models and use cases. DynamoDB is the dominent leader in NoSQL use cases, which bring great values to customer. Let's take Amazon itself as an example. Amazon consumer business successfully turned off all the 7500 'one-size-fits-all' Oracle database with 75 petabytes data in 2019. The benifit is huge: 60% cost savings, 40% performance improvements and 70% administrative overhead reduce.

More customers have the opportunity to differentiate and identify NoSQL scenarios and migrate from legacy SQL database to AWS DynamoDB by using MonoSQL.

MonoSQL Overview

Amazon DynamoDB is a fully managed, serverless, key-value NoSQL database designed to run high-performance applications at any scale. DynamoDB is suitable for NoSQL senarios and has huge adoptions worldwide by industries like Gaming, Retail, Social Media, Entertainment, Transportation etc.

But migration from SQL database to DynamoDB is difficult for customers and often introduce additional dev cost and schema migration cost. To lower the barriers, MonoSQL is designed to become the SQL wrapper of DynamoDB. Benefiting from DynamoDB's serverless architecture and single-digit millisecond performance at any scale, MonoSQL brings the extreme scalability and high concurrent write throughput to SQL ecosystem, while offers a low-cost on-demand billing model. In addition, MonoSQL also has powerful security and high availability features, which can ensure data security and enable your applications to stay highly available even in the rare event of isolation or degradation of an entire Region.

MonoSQL's key features includes:

1. Full Managed. MonoSQL offloads operational burden for customer.
2. Scalability. MonoSQL enables fast performance while scaling to any workload
3. Flexible. MonoSQL supports fast schema change.
4. SQL Support. MonoSQL supports JDBC/ODBC drivers, basic transactions and complete SQL query API incuding Join, Aggregation and Recursive CTE etc.
5. Security. MonoSQL has full access control like SQL and supports encryption at rest.
6. High Availability. MonoSQL supports global tables in DynamoDB which survives region failure.